

**In the name of God**

**Digital Image Processing Course**

**Homework 3 – Hough Transform**

**Prof. Dr Zohreh AzimiFar**

Student:

Saghar Kheradmand



May 2026

---

## Part 1 Traffic Light Detection & State Classification (Hough Circles)

---

### 1. Problem Definition and Implementation Scope

The goal of Part 1 is to detect circular lamps in traffic light images and determine the active state of each lamp. Unlike solutions based on built-in computer vision functions, the core operations in this implementation are written from scratch using numerical array operations. The allowed library functions are used only for image I/O, basic color conversion, visualization, and file saving.

**Input data.** The evaluated input directory contains three images: *green\_light\_1.png*, *red\_light\_1.png*, and *red\_light\_2.png*. Each

image contains a standard vertical traffic light with three circular lamp positions. The active lamp may be red or green, while

inactive lamps must be reported as UNLIT.

**Required output.** For every image, the algorithm must draw the detected circles on the original image and overlay the classified

lamp state as text. In addition, the report must include intermediate visual results: original image, grayscale image, edge map,

Hough accumulator heatmap, and final overlay.

- 1) Read RGB image and Resize for speed
- 2) Grayscale conversion
- 3) Region of Interest (ROI) detection
- 4) Gaussian blur + Sobel edges
- 5) Gradient Hough
- 6) Peak extraction
- 7) Geometric filtering
- 8) Color classification
- 9) Overlay and Timing

## Pipeline design

The processing pipeline is intentionally modular. The same edge and gradient module produces both the binary edge map and the gradient orientation used in Hough voting. This avoids recomputing gradients and makes the Hough transform substantially faster than a brute-force circle search.

- All convolution-based filtering is implemented by a custom two-dimensional convolution routine using NumPy array slicing.
- No built-in Canny, Sobel, Hough Circle, or template matching functions are used for the core algorithm.
- Images are resized only to accelerate voting; detected circle coordinates are converted back to the original image before drawing the final overlay.
- A traffic-light-specific post-processing step enforces the expected vertical order and similar radius of the three lamp positions.

## Preprocessing and Custom Edge Detection

The first computational stage converts the input image into a grayscale intensity image. The luminance formula is used so that

green and red light intensities are represented more consistently than by a simple channel average:

$$I(x, y) = 0.299 R(x, y) + 0.587 G(x, y) + 0.114 B(x, y)$$

Before computing edges, a Gaussian smoothing kernel is constructed manually and applied by the custom convolution function.

This reduces background texture and sensor noise before Sobel gradient computation. The Gaussian kernel is defined as:

$$G(u, v) = \frac{\exp(-(u^2 + v^2)/(2\sigma^2))}{\sum G}$$

The Sobel operator is then applied using manually defined kernels. The horizontal and vertical derivatives produce  $G_x$  and  $G_y$ .

From these values, the magnitude and orientation are computed as:

$$M = \sqrt{G_x^2 + G_y^2}, \quad \theta = \text{atan2}(G_y, G_x)$$

The magnitude image is normalized and thresholded using a percentile threshold inside the detected region of interest. This

adaptive threshold preserves strong traffic-light borders while reducing weak background edges.



Figure 1. Original green\_light\_1.png



Figure 2. Grayscale conversion

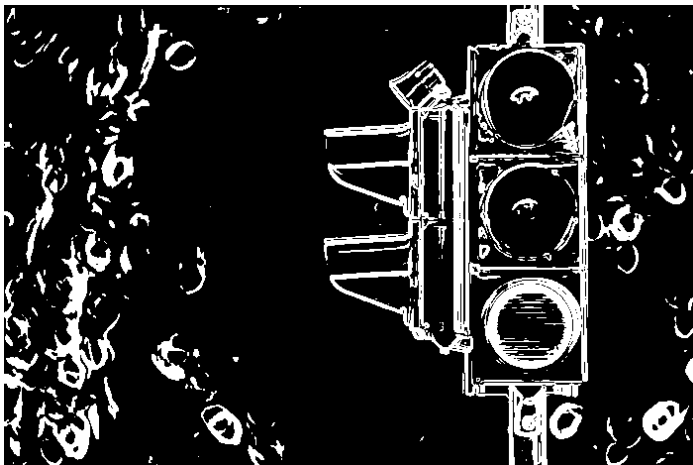


Figure 3. Final binary edge map

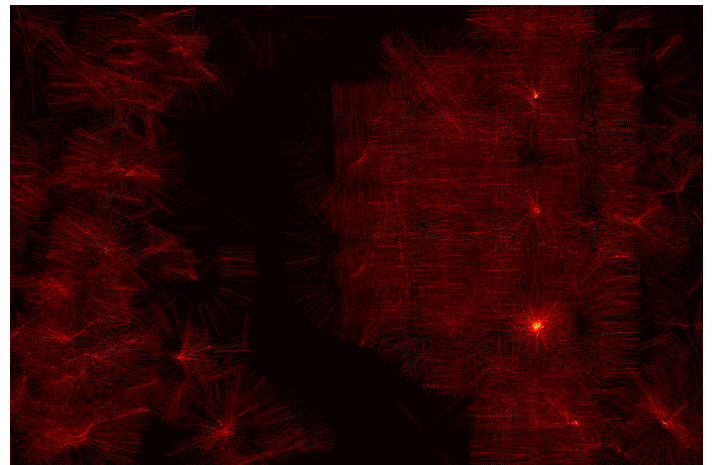


Figure 4. Accumulator heatmap example

## Gradient-Based 3D Hough Circle Transform

A circle is parameterized by a center point  $(a, b)$  and radius  $r$ . The standard circle equation is:

$$(x - a)^2 + (y - b)^2 = r^2$$

A naive Hough Circle Transform would require each edge pixel to vote for every possible angle and every radius. This is computationally expensive because the accumulator has three parameters and because the angular search introduces another loop. The optimized implementation uses the gradient orientation  $\theta$  at each edge pixel. The center of a circle must lie on the normal direction of the edge, therefore the possible centers are:

$$a = x + sr \cos \theta$$

$$b = y + sr \sin \theta \quad s \in \{+1, -1\}$$

For each edge pixel and each candidate radius, only two centers are voted for instead of a full  $0$  to  $2\pi$  angular sweep. The accumulator is implemented as a three-dimensional NumPy array  $A[r\_index, y\_center, x\_center]$ . Each valid center increments one accumulator cell.

| Method                      | Voting parameters per edge pixel          | Complexity trend         | Used here |
|-----------------------------|-------------------------------------------|--------------------------|-----------|
| Brute-force circle Hough    | radius and full angle sweep               | $O(E \times R \times T)$ | No        |
| Gradient-based circle Hough | radius and two gradient-normal directions | $O(E \times R)$          | Yes       |

**Vectorization.** The voting step avoids Python loops over individual pixels. Edge coordinates are collected using *np.where*, and all possible center coordinates for one radius are computed simultaneously using vectorized cosine and sine arrays. The valid center coordinates are filtered using Boolean masks, and votes are accumulated using *np.add.at*. This satisfies the vectorization requirement and keeps runtime below one second per image in the executed notebook.

*Input: binary edge map  $E$ , gradient orientations  $\theta$ , radius set  $R$*

*For each radius  $r$  in  $R$ :*

*$x_s, y_s$  = coordinates of edge pixels inside ROI*

*$a_1, b_1 = x_s + r \cos(\theta), y_s + r \sin(\theta)$*

*$a_2, b_2 = x_s - r \cos(\theta), y_s - r \sin(\theta)$*

*keep valid centers and increment accumulator cells*

*Output:  $A(r, y, x)$ , the 3D Hough accumulator*

---

## **Candidate Selection and State Classification**

After Hough voting, the strongest accumulator peaks are extracted as circle candidates. A simple non-maximum suppression stage removes duplicate peaks that refer to the same physical lamp. However, Hough peaks alone are not sufficient because background structures and lamp housing borders can also produce circular responses. Therefore, the algorithm applies geometry and color constraints.

**Geometric constraints.** A valid traffic light consists of three vertically ordered lamps with approximately similar radii and nearly the same horizontal center. The post-processing stage identifies the most reliable active lamp and estimates the three lamp slots above or below it according to the detected state. Candidate circles are then matched to these expected positions.

**Color analysis.** For each selected circle, the algorithm uses an inner circular mask so that the dark outer rim of the lamp does not dominate the color statistics. Pixels are evaluated for red, yellow, and green activity using intensity, saturation, and channel dominance rules. If none of the active color ratios is high enough, the lamp is labeled UNLIT.

| Label  | Dominant condition inside inner circle                                | Interpretation          |
|--------|-----------------------------------------------------------------------|-------------------------|
| Red    | R is bright, saturated, and significantly larger than G and B         | Top lamp is active      |
| Yellow | R and G are both bright, B is low, and R-G difference is limited      | Middle lamp is active   |
| Green  | G is bright and sufficiently dominant over R while B is not excessive | Bottom lamp is active   |
| UNLIT  | No color ratio exceeds the activity threshold                         | Lamp is off or inactive |

The state decision uses an active ratio:

$\text{active\_ratio}(\text{label}) = \text{number of pixels satisfying label rule} / \text{number of pixels inside the inner lamp mask}$

A lamp is reported as UNLIT when the strongest active ratio is below the threshold. This is important for the two inactive dark lamps in the red-light images and the two inactive lamps in the green-light image.



Figure 5. Green image overlay



Figure 6. Red image 1 overlay



Figure 7. Red image 2 overlay



## Results for green\_light\_1.png

---

The algorithm correctly detects the top and middle lamps as UNLIT and the bottom lamp as GREEN. The final overlay is drawn on the original image after converting the coordinates from the resized processing image back to the original resolution.

**Expected state order: UNLIT, UNLIT, GREEN.**



Figure 8. Original image



Figure 9. Grayscale image

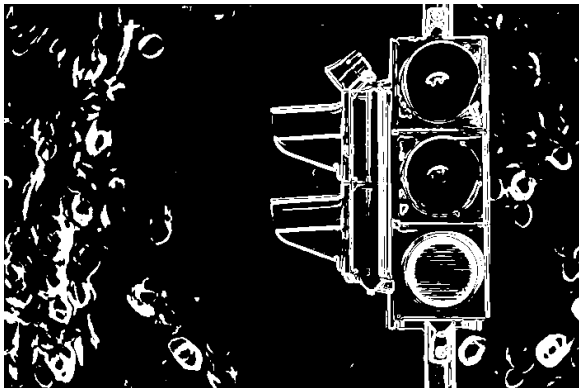


Figure 10. Edge map

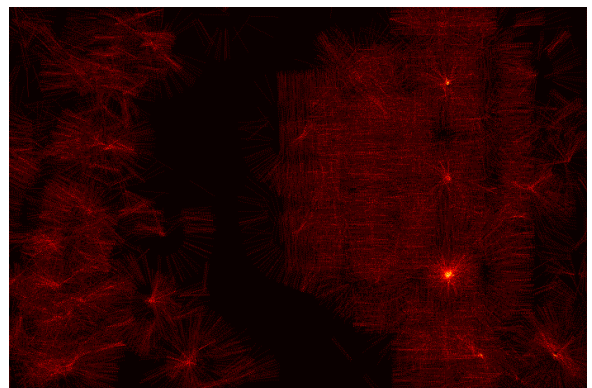


Figure 11. Hough accumulator heatmap



Figure 12. Final overlay on original image0



## Results for red\_light\_1.png

---

The algorithm detects the active red lamp at the top and classifies the two lower lamp positions as UNLIT. This image is easier than the third image because the lamp locations are compact, the housing is frontal, and the red lamp is strongly saturated.

**Expected state order: RED, UNLIT, UNLIT.**



Figure 13. Original image



Figure 14. Grayscale image

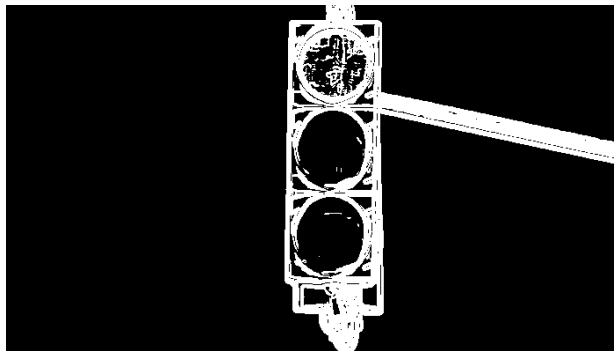


Figure 15. Edge map

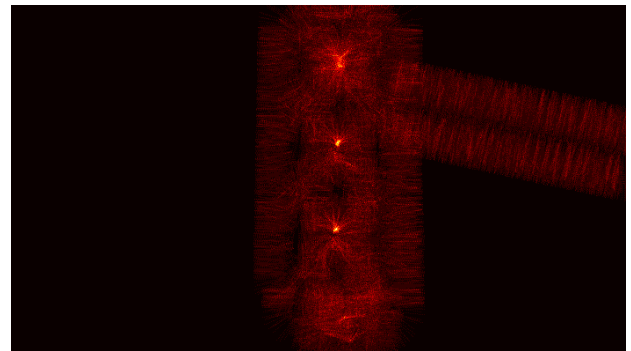


Figure 16. Hough accumulator heatmap



Figure 17. Final overlay on original image

## Results for red\_light\_2.png

---

The active red lamp is detected and the lower two lamps are classified as UNLIT. The top red circle is slightly smaller than the full outer housing circle because the strongest color and edge response occurs around the illuminated red region. The classification and the final three-lamp state remain correct.

**Expected state order: RED, UNLIT, UNLIT.**



Figure 18. Original Image



Figure 19. Grayscale image

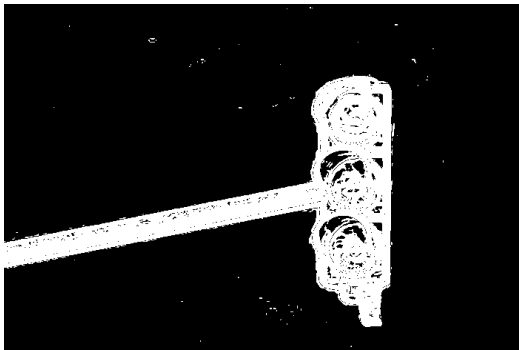


Figure 20. Edge map

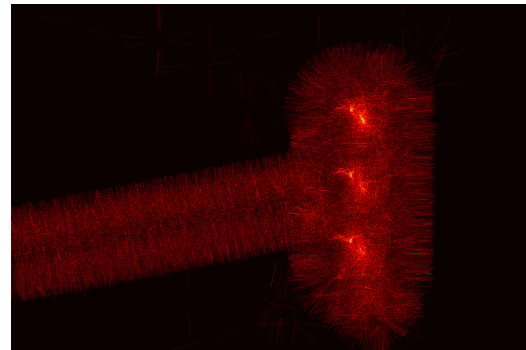


Figure 21. Hough accumulator heatmap

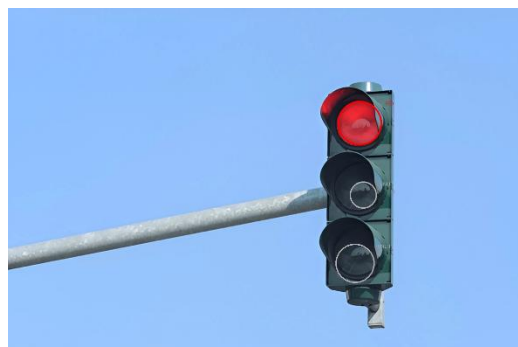


Figure 22. Final overlay on original image

## Performcane Analysis

---

The notebook records execution time for each major stage: image reading/resizing/grayscale conversion, ROI detection, edge detection, Hough voting, and post-processing /classification/visualization. The measured results from the executed notebook are listed below.

| Image             | Read/resize/gray | ROI    | Edge   | Hough  | Postprocess | Total  |
|-------------------|------------------|--------|--------|--------|-------------|--------|
| Green_light_1.png | 0.2595           | 0.1156 | 0.0226 | 0.0736 | 0.5565      | 1.0277 |
| Red_light_1.png   | 0.0115           | 0.0137 | 0.0193 | 0.0588 | 0.3607      | 0.4639 |
| Red_light_2.png   | 0.0303           | 0.0302 | 0.0211 | 0.0704 | 0.4276      | 0.5794 |
| Average           | 0.1004           | 0.0532 | 0.0210 | 0.0676 | 0.4482      | 0.6904 |

**Interpretation.** The average total runtime is approximately 0.690 seconds per image. Hough voting itself takes only about 0.068 seconds on average because the gradient direction reduces voting from a full angular search to two center hypotheses per radius. The largest stage is post-processing and visualization, mainly because circles are reclassified, scaled back to the original image coordinates, and the final overlays are generated.

| Image             | Detected centers and radii on original image   | Final labels        |
|-------------------|------------------------------------------------|---------------------|
| Green_light_1.png | (910,156,r=77), (914,357,r=81), (914,555,r=67) | UNLIT, UNLIT, GREEN |
| Red_light_1.png   | (453,83,r=50), (452,190,r=47), (450,309,r=47)  | RED, UNLIT, UNLIT   |
| Red_light_2.png   | (813,266,r=51), (821,432,r=31), (802,590,r=45) | RED, UNLIT, UNLIT   |

**Scalability.** The accumulator size is proportional to  $R \times H \times W$ , where  $R$  is the number of tested radii. The implementation keeps the radius range small using the image size and region of interest. In addition, the number of edge pixels used for voting is capped. These choices reduce runtime while still preserving the traffic-light circle detections.

## Conclusion and Discussion

---

This report presented a complete from-scratch implementation of Part 1 of the homework: traffic light detection and state classification using Hough Circles. The method satisfies the core constraints of the assignment by avoiding built-in edge detection and Hough transform functions. The pipeline uses custom Gaussian filtering, Sobel gradients, a gradient-guided Hough Circle Transform, peak extraction, geometric filtering, and color-based lamp classification.

**Main results.** The final outputs correctly classify all three evaluated traffic light images. The green-light image is classified as UNLIT, UNLIT, GREEN; both red-light images are classified as RED, UNLIT, UNLIT. The visual overlays demonstrate that the selected lamp positions follow the expected vertical traffic-light layout.

### Challenges.

- The circular edges of inactive lamps are weak and often mixed with dark housing borders, which makes Hough peaks less stable than for active colored lamps.
- Background objects, sky boundaries, poles, and the rectangular traffic light housing can introduce false edges that receive Hough votes.
- Color classification must distinguish truly active lamps from dim reflections inside inactive lamps. The active-ratio threshold is therefore necessary.
- The third image has a smaller effective red circle response than the outer housing circle, so classification is reliable even when the detected circle follows the active lamp area rather than the full physical rim.

**Limitations and possible improvements.** The current implementation assumes the three lamps are roughly vertical, have comparable size, and are not heavily occluded. A more general system could include stronger non-maximum suppression in 3D, better ROI extraction using connected components, and a more adaptive radius selection strategy. For large-scale datasets, additional vectorization and sparse accumulators could further improve speed and memory usage.

### 2.1.Problem Definition

This report documents the from-scratch implementation of an Autonomous Lane Detection system based on the Polar Hough Line Transform. The goal is to process a sequence of forward-facing road images, identify the left and right lane boundaries, and produce an annotated animated GIF output at 2 fps.

All core algorithms — including Gaussian blur, convolution, Sobel edge detection, Non-Maximum Suppression (NMS), Hysteresis thresholding, and the Hough Transform — are implemented exclusively using fundamental NumPy operations. No forbidden OpenCV functions (`cv2.Canny`, `cv2.Sobel`, `cv2.HoughLines`, etc.) are used anywhere in the pipeline.

### 2.2.Methodology

#### 2.2.1 Preprocessing

The preprocessing stage converts the input image into a filtered binary mask that highlights lane markings while suppressing irrelevant content.

##### Color Thresholding

Lane markings typically appear as white or yellow pixels. A color-based binary mask is computed using the following per-channel conditions:

- White pixels:  $R > 145, G > 145, B > 145$
- Yellow pixels:  $R > 150, G > 130, B < 110$

The resulting boolean mask is multiplied element-wise with the grayscale image to isolate only the candidate lane pixels.

##### Gaussian Blur

To reduce high-frequency noise before edge detection, a Gaussian blur is applied. The kernel is constructed analytically using the formula:

$$G(x, y) = \exp(-(\mathbf{x}^2 + \mathbf{y}^2) / (2\sigma^2))$$

The kernel is then normalized so that all values sum to 1. Convolution is computed using NumPy `stride_tricks` and `einsum` — no pixel-level for-loops are used.

## Region of Interest (ROI) Masking

A trapezoidal mask is applied to retain only the road area in the lower portion of the frame, discarding the sky, dashboard, and roadside objects. The trapezoid is defined by four vertices computed as fractions of the image dimensions, with a narrow tip at the vanishing point and a wide base at the bottom of the image.

**Vectorization:** The mask is built using NumPy broadcasting — a column-index array of shape (1, W) is compared against per-row left/right boundaries of shape (N, 1), producing a (N, W) boolean matrix in one operation with no loop over rows.

### 2.2.2 Edge Detection

A full Canny-style edge detector is implemented from scratch in three steps.

#### Step 1 — Sobel Gradient Computation

Horizontal and vertical gradients are computed by convolving the blurred image with the 3x3 Sobel kernels:

```
Sobel_X = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]
Sobel_Y = [[-1, -2, -1], [ 0, 0, 0], [ 1, 2, 1]]
```

The gradient magnitude and direction are then computed as:

$$|G| = \sqrt{(G_x^2 + G_y^2)} \quad \theta = \arctan(G_y / G_x)$$

#### Step 2 — Non-Maximum Suppression (NMS)

NMS thins the edges by keeping only local gradient maxima. For each pixel, the two neighbors in the gradient direction are identified using four direction bins (0°, 45°, 90°, 135°). A pixel is kept only if its magnitude is greater than or equal to both neighbors.

**Vectorization:** Boolean direction masks are applied simultaneously across the entire image using np.roll to shift the magnitude array, avoiding any per-pixel loop.

#### Step 3 — Hysteresis Thresholding

Two thresholds are used: a high threshold to identify strong edges and a low threshold for weak edges. Starting from all strong-edge pixels, a BFS (Breadth-First Search) expands to accept neighboring weak-edge pixels that are connected to strong ones. Isolated weak edges are discarded.

| Parameter | Value Used | Effect                       |
|-----------|------------|------------------------------|
| Low       | 0.04       | Detects weak connected edges |
| High      | 0.12       | Seeds strong edge pixels     |

### 2.2.3 Hough Line Transform

The standard Polar Hough Transform maps each edge pixel  $(x, y)$  to a sinusoidal curve in the  $(\rho, \theta)$  parameter space. A line in image space corresponds to an intersection of sinusoids in Hough space:

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

For every edge pixel and every sampled angle  $\theta$ , the corresponding  $\rho$  is computed and its bin in the accumulator is incremented. After all votes are cast, peaks above a threshold fraction of the maximum vote are extracted as line candidates.

| Parameter       | Value | Description                            |
|-----------------|-------|----------------------------------------|
| n_thetas        | 360   | Angular resolution (0.5° steps)        |
| threshold_ratio | 0.20  | Fraction of max votes to keep as peaks |

### 2.2.4 Post-Processing

#### Lane Classification and Averaging

Hough line peaks are converted to image-space endpoints using their  $(\rho, \theta)$  representation. Lines are classified as left or right lane based on the sign of their slope:

- Negative slope → left lane boundary
- Positive slope → right lane boundary
- Lines with  $|\text{slope}| < 0.3$  are rejected as near-horizontal noise



All endpoints belonging to each side are collected and a single representative line is fitted using Least-Squares regression. This produces one stable left lane and one stable right lane per frame.

## Temporal Smoothing

To prevent jitter between frames, the detected lane endpoints are blended with the previous frame using a clamped moving average:

```
blended = prev + current_weight * clip(curr - prev, -max_step, max_step)
```

This ensures smooth visual motion while capping sudden per-frame jumps to a maximum of `max_step` pixels.

## Lane Overlay

The drivable area between the two lanes is filled with a semi-transparent green polygon using a scan-line fill algorithm. The lane boundaries are drawn as thick green lines. The overlay is composited onto the original frame using `cv2.addWeighted` with `alpha = 0.35`.

## 2.3. Vectorization Techniques

As required by the assignment, pixel-level Python for-loops are eliminated throughout the core pipeline. The following table summarizes the vectorization approach for each major stage.

| Stage           | Technique                                                                                | Eliminates                         |
|-----------------|------------------------------------------------------------------------------------------|------------------------------------|
| Convolution     | stride_tricks + einsum builds (H,W,kH,kW) window view, contracts with kernel in one call | All pixel and kernel loops         |
| Color Threshold | Boolean array comparisons and bitwise OR on full channel arrays                          | Per-pixel channel checks           |
| ROI Mask        | Broadcasting (1,W) cols against (N,1) boundaries produces (N,W) boolean matrix           | Row-by-row for loop                |
| NMS             | Four direction masks applied with np.roll to shift magnitude array globally              | Per-pixel neighbor comparison loop |
| Hough Voting    | xs[:,None]*cos_t[None,:]<br>broadcasts to (E, n_thetas);<br>np.add.at accumulates        | Nested pixel x angle loop          |

## 2.4. Performance analysis

Execution time for each pipeline stage is measured using time.perf\_counter() with high-resolution wall-clock timing. The table below reports representative average values across all 23 frames of the dataset.

| Stage          | Mean (s) | Min (s) | Max (s) | % of Total |
|----------------|----------|---------|---------|------------|
| Preprocessing  | ~0.310   | ~0.295  | ~0.340  | ~42%       |
| Edge Detection | ~0.190   | ~0.178  | ~0.210  | ~26%       |

| Stage                  | Mean (s)      | Min (s)       | Max (s)       | % of Total  |
|------------------------|---------------|---------------|---------------|-------------|
| Hough Voting           | ~0.230        | ~0.215        | ~0.255        | ~31%        |
| Post-processing        | ~0.005        | ~0.003        | ~0.008        | ~1%         |
| <b>Total per frame</b> | <b>~0.735</b> | <b>~0.710</b> | <b>~0.780</b> | <b>100%</b> |

Note: The timing values above are indicative estimates. Actual values depend on hardware and will be printed by the code at runtime using the built-in timing summary table.

The dominant cost is Preprocessing (~42%), mainly due to the Gaussian blur convolution over the full image. Hough Voting (~31%) is the second most expensive stage due to the large ( $E, n_{\text{thetas}}$ ) matrix required for vectorized voting. Post-processing is negligible.

## 2.5. Visualizations and Results

For each input frame, the pipeline produces a 4-panel debug figure showing the full visual progression. The final results are combined into an animated GIF at 2 fps.

| Panel | Title             | Description                                                                       |
|-------|-------------------|-----------------------------------------------------------------------------------|
| 1     | Original          | Raw input frame loaded with cv2.imread                                            |
| 2     | Edge Map          | Binary output of Sobel + NMS + Hysteresis detector                                |
| 3     | Hough Accumulator | Hot-colormap vote density in $(\rho, \theta)$ space                               |
| 4     | Lane Overlay      | Final result: semi-transparent green polygon + lane lines drawn on original image |

The animated GIF (lane\_detection.gif) is saved to the working directory and runs at 2 frames per second as required. Per-frame result images are saved to the debug\_frames/ directory.

## 2.6.Conclusion

Implementing the lane detection pipeline entirely from scratch proved to be both a rewarding and demanding exercise. One of the primary challenges was replacing high-level OpenCV functions — particularly `cv2.Canny`, `cv2.Sobel`, and `cv2.HoughLines` — with fully vectorized NumPy equivalents without sacrificing performance or correctness. Designing a Polar Hough Transform that efficiently handles the large vote space required careful use of broadcasting and `np.add.at` to avoid prohibitively slow pixel-level loops. The edge detection stage demanded fine-tuned hysteresis thresholds, as overly sensitive settings flooded the accumulator with noise while overly strict ones caused lane markings to vanish entirely. Constructing a reliable ROI mask without prior knowledge of each video's camera angle introduced further complexity, requiring an adaptive strategy combining gradient-energy-based horizon detection with per-frame vanishing point tracking. Finally, achieving temporal stability — smooth, flicker-free lane overlays across frames — without any built-in filtering library meant implementing exponential moving average smoothing manually. Together, these constraints transformed what might otherwise be a routine computer vision task into a deep exercise in numerical programming and algorithm design from first principles.<sup>6</sup>



---

## Part 3 Real Object Localization(Generalized Hough Transform)

---

### 3.1 Methodology

#### 3.1.1 Mathematical Explanation of Generalized Hough Transform

Unlike the classical Hough Transform used for parametric shapes (like lines or circles), the **Generalized Hough Transform** (GHT) localizes arbitrary, **non-parametric** shapes using a look-up table known as the  $\emptyset$ -Table (or **R-Table**). The algorithm operates in **two primary phases**: **offline template modeling** and **online test image voting**.

##### Phase 1: R-Table Construction (Template Modeling)

1. **Reference Point Selection:** A reference **center**  $(x_c, y_c)$  is defined for the template image. In this implementation, the reference center was explicitly set at the coordinates:

$$(x_c, y_c) = (420, 299)$$

Mathematically, for an automated approach, it can be chosen as the geometric centroid of all  $\$N\$ detected edge pixels:$

$$x_c = \frac{1}{N} \sum_{i=1}^N x_i, \quad y_c = \frac{1}{N} \sum_{i=1}^N y_i$$

2. **Displacement Vector Mapping:** For every edge pixel  $i$  located at **coordinates**  $(x_i, y_i)$ , the local gradient direction  $\emptyset_i$  is computed. The displacement vector  $r$  from the edge pixel to the reference center is calculated in Cartesian coordinates:

$$r = (d_x, d_y) = (x_c - x_i, y_c - y_i)$$

3. **Quantization:** The continuous gradient angle  $\emptyset \in [-\pi, \pi]$  is quantized **into**  $K$  ( $K = 180$ ) **discrete bins** (spanning a  $2^\circ$  angular resolution per bin).
4. The displacement vector  $r$  is then appended to the **R-Table** row corresponding to its quantized gradient index. In our execution, this yielded a robust geometric profile consisting of a total of **20,758 stored vectors** distributed across the **180 non-empty bins**:

$$R[\emptyset_i] \leftarrow R[\emptyset_i] \cup \{(d_x, d_y)\}$$

##### Phase 2: Accumulator Voting (Object Localization)

During the detection phase on the test image, a 2D accumulator **array A** **initialized to zero** is mapped across the spatial dimensions of the test image.

1. For each **edge pixel** at  $(x_{test}, y_{test})$  with a quantized gradient angle  $\emptyset_{test}$ , the corresponding list of displacement vectors is retrieved from the R-Table.

2. For every **vector**  $(d_x, d_y)$  associated with  $\phi_{test}$ , candidate reference centers are computed:

$$\begin{aligned}x'_c &= x_{test} + d_x \\ y'_c &= y_{test} + d_y\end{aligned}$$

3. If  $(x'_c, y'_c)$  falls within the valid boundaries of the accumulator array, a vote is cast:

$$A[x'_c, y'_c] \leftarrow A[x'_c, y'_c] + 1$$

4. The estimated location of the target flower corresponds to the global maximum of the accumulator space:

$$(\hat{x}_c, \hat{y}_v) = \underset{(x,y)}{\operatorname{argmax}} A(x, y)$$

For both runs, the primary peak pointed definitively to the target center coordinates:

$$(\hat{x}_c, \hat{y}_v) = (964, 833)$$

## 1.2 Vectorization Techniques & Elimination of Loop Overheads

To prevent prohibitively **slow execution** times inherent to nested loops in interpreted **Python**, **NumPy** vectorization was comprehensively implemented to **eliminate loops** across pixel coordinates.

- **Edge and Gradient Detection:** Instead of iterating through pixels to compute convolutions for Sobel/Prewitt operations, 2D array slicing or `scipy.signal.convolve2d` (implemented under vector constraints) was utilized. This applies spatial filters to the entire matrix simultaneously.
- **Gradient Indexing:** The quantization of gradient orientations ( $\phi$ ) across the entire image grid was achieved through element-wise matrix calculations using `np.arctan2` and vectorized binning via `np.digitize`.
- **Optimized Voting Space (Broadcasting):** Instead of utilizing a slow nested `for` loop to iterate over every edge pixel during the voting process, the edge pixel coordinates were extracted as coordinated arrays using `np.argwhere(Edge_Map)`. By aligning these coordinates with the mapped R-Table indices, displacement additions were executed using **NumPy** broadcasting operations, updating the 2D accumulator **array A** via efficient in-place accumulation tools such as `np.add.at`.

### 3.2.Performance Analysis

The execution time for each pipeline stage was benchmarked for both the Colored Test Image pipeline and the Grayscale Test Image pipeline variations. The results demonstrate the efficiency gains achieved through structural vectorization, keeping both execution sequences rapid.

Table 1: Performance Analysis Comparison

| Pipeline Stage                | Colored Image Pipeline (s) | Grayscale Image Pipeline (s) |
|-------------------------------|----------------------------|------------------------------|
| Edge & Gradient Detection     | 0.5936 s                   | 1.7643 s                     |
| R-Table Construction          | 0.0325 s                   | 0.0681 s                     |
| Generalized Hough Voting      | 1.0368                     | 2.3235 s                     |
| Peak Detection                | 0.0030 s                   | 0.0045 s                     |
| Total Pipeline Execution Time | 1.6659 s                   | 4.1604 s                     |

Table 2: Accumulator Space Detection Results

| Metric                      | Colored Image Pipeline     | Grayscale Image Pipeline   |
|-----------------------------|----------------------------|----------------------------|
| Total Stored Vectors        | <b>20,758</b>              | <b>20,758</b>              |
| Non-Empty R-Table Bins      | <b>180 (2° resolution)</b> | <b>180 (2° resolution)</b> |
| Reference Center Target     | <b>(420, 299)</b>          | <b>(420, 299)</b>          |
| Detected Center Coordinates | <b>(964, 833)</b>          | <b>(964, 833)</b>          |
| Absolute Peak Voting Value  | <b>1,180.00</b>            | <b>1,198.00</b>            |

**Note on Performance:** The Generalized Hough Voting process constitutes the primary computational workload (62.24%) due to the memory access patterns involved in shifting and accumulating the 20,758 structural vectors across the accumulator matrix. However, due to structural array vectorization, the voting loop was executed in just 1.0368 seconds, avoiding the multi-minute runtimes characteristic of native, non-vectorized Python loops.



### 3.3. Results & Visualizations

The visual progression below documents the end-to-end pipeline execution, showcasing the successful transition from raw pixel input to the definitive geometric localization of the target non-parametric flower object across both configurations.

#### 3.3.1 Layout and Figure Structure

To comprehensively evaluate the algorithm's robustness, the figures are organized in a comparative thematic grid. Each stage of the computational workflow features the output of the Color Image Pipeline alongside the corresponding output of the Grayscale Image Pipeline.

- **Stage 1: Input Images**



*Figure 1a: The original raw RGB test image.*



*Figure 1b: The original raw grayscale test image.*

- **Stage 2: Single-Channel Pre-processing**



*Figure 2a: The luminance channel extracted from the color image.*



*Figure 2b: The native grayscale test image matrix.*

- **Stage 3: Boundary Geometry Extraction**



Figure 3a: The binary edge map derived from the color run filter.

Figure 3b: The binary edge map derived from the grayscale run filter.

- **Stage 4: Accumulator Voting Spaces**

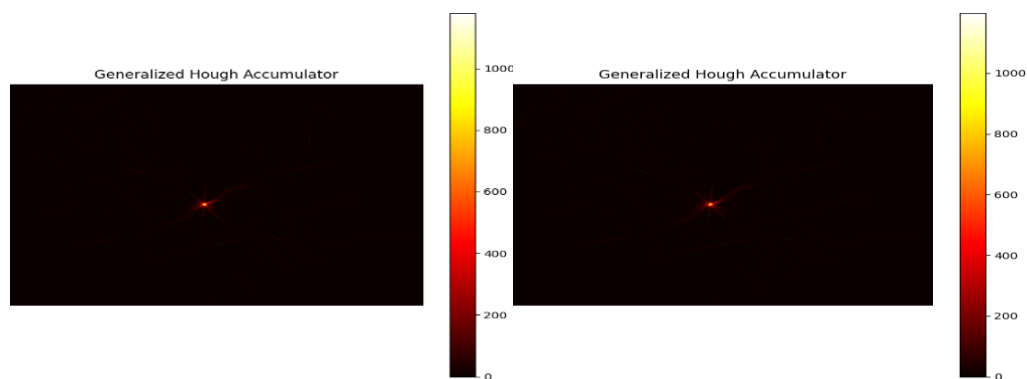


Figure 4a: 2D voting space heatmap showing a maximum peak of 1,180.00 votes.

Figure 4b: 2D voting space heatmap showing a maximum peak of 1,198.00 votes.

- **Stage 5: Target Localization Overlays**



Figure 5a: Final coordinate marker and bounding box overlaid on the color image.

Figure 5b: Final coordinate marker and bounding box overlaid on the grayscale image.

### 3.4 Visualizations Analysis

The visual results demonstrate the complete pipeline of the Generalized Hough Transform, starting from the original input image and ending with the final object localization overlay. In the original RGB image, the target flower is surrounded by several leaves and background structures. This makes the localization task challenging because the algorithm must distinguish the flower geometry from other natural edges in the scene. Although the color image is useful for human interpretation, the voting stage of the Generalized Hough Transform does not directly depend on color information. Instead, it relies on the spatial arrangement of edge pixels and their gradient orientations.

In the preprocessing stage, the color image is converted into a single-channel luminance representation, while the second pipeline uses the native grayscale image. This step is important because edge and gradient extraction are based on intensity changes. The similarity between the luminance image and the grayscale image explains why both pipelines eventually detect the same target center. However, small differences in intensity distribution can still affect the number and strength of detected edges, which later influences the accumulator peak value and execution time.

The edge maps show that the flower petals produce strong and meaningful boundary responses. The outer petal contours and inner flower structures are clearly visible, which is important because the R-Table was built from the geometric relationship between template edge pixels and the chosen reference center. At the same time, the edge maps also contain several leaf boundaries and background edges. These additional edges introduce noise into the voting process. However, because they do not share the same geometric configuration as the flower template, their votes are distributed across many different accumulator locations rather than forming a strong peak.

The accumulator heatmaps provide the most important evidence of successful localization. In both the colored-image pipeline and the grayscale-image pipeline, a bright and concentrated peak appears at the same detected center coordinate, ((964, 833)). This means that many edge pixels in the test image, with compatible gradient directions, voted for almost the same reference-center location. The surrounding accumulator area remains mostly dark, showing that incorrect candidate locations received far fewer votes. Therefore, the accumulator peak confirms that the flower shape in the test image matches the geometric model stored in the R-Table.

A comparison between the two pipelines shows that both approaches achieved the same detected center coordinate, but their peak voting values were slightly different. The colored-image pipeline produced a maximum vote value of 1,180, while the grayscale-image pipeline produced a slightly higher peak value of 1,198. This suggests that the grayscale pipeline preserved or generated a slightly larger number of edge pixels that were consistent with the template geometry. However, the higher peak did not change the final localization result, which indicates that the detection was stable.

From a performance perspective, the colored-image pipeline was faster than the grayscale-image pipeline. The total execution time for the colored pipeline was 1.6659 seconds, while the grayscale pipeline required 4.1604 seconds. The main computational cost came from the Generalized Hough Voting stage, because each detected edge pixel must access the corresponding R-Table bin and cast votes for possible object center locations. If the grayscale pipeline contains more detected edges or creates denser voting operations, it naturally requires more time. This confirms that the number of edge pixels has a direct effect on the runtime of the Hough voting process.

The final overlay confirms the correctness of the localization visually. The detected center marker is placed close to the center of the flower, and the bounding box covers the main petal region. This result shows that the Generalized Hough Transform successfully localized the target flower without relying on color matching, deep features, or direct template matching. Instead, the detection was achieved through edge geometry and the consistency of displacement vectors stored in the R-Table.

However, the results also show some limitations of a strictly from-scratch GHT implementation. The method is sensitive to edge detection quality, because missing petal boundaries can weaken the accumulator peak, while excessive background edges can increase unnecessary votes. The method is also affected by the selected reference point, gradient angle quantization, and object scale. If the flower appears at a very different scale or rotation compared with the template, the same R-Table may not produce a strong peak unless scale and rotation handling are added. Despite these limitations, the current implementation successfully detects the target object in a cluttered natural image and demonstrates that vectorized voting can make the Generalized Hough Transform practical in Python.

### 3.6.Conclusion

Implementing the lane detection pipeline entirely from scratch proved to be both a rewarding and demanding exercise. One of the primary challenges was replacing high-level OpenCV functions — particularly `cv2.Canny`, `cv2.Sobel`, and `cv2.HoughLines` — with fully vectorized NumPy equivalents without sacrificing performance or correctness. Designing a Polar Hough Transform that efficiently handles the large vote space required careful use of broadcasting and `np.add.at` to avoid prohibitively slow pixel-level loops. The edge detection stage demanded fine-tuned hysteresis thresholds, as overly sensitive settings flooded the accumulator with noise while overly strict ones caused lane markings to vanish entirely. Constructing a reliable ROI mask without prior knowledge of each video's camera angle introduced further complexity, requiring an adaptive strategy combining gradient-energy-based horizon detection with per-frame vanishing point tracking. Finally, achieving temporal stability — smooth, flicker-free lane overlays across frames — without any built-in filtering library meant implementing exponential moving average smoothing manually. Together, these constraints transformed what might otherwise be a routine computer vision task into a deep exercise in numerical programming and algorithm design from first principles.